

Comandos básico do Shell do Linux ➡

INDEX

1. Introdução

2. Atalho para criar um terminal de comandos

3. Comandos de Gestão de Arquivos e Pastas

1. *ls* - Comando para listar arquivos e pastas
2. *which* - mostra o caminho completo dos comandos
3. *cd* - Comando para mover-se entre os diretórios
4. *ln* - Comando para criar links simbólico
5. *cp* - Comando para copiar arquivos ou pastas
6. *dd* - Clona um dispositivo de bloco, convertendo e formatando de acordo com os operandos.
7. *e2image* - Clona um dispositivo de bloco cujo o formato de arquivo seja ext2, ext3 e ext4
8. *gparted* - Programa usado para criar partição, pode ser usado para clonar partição de forma muito simples e visual
9. *mount* - monta um sistema de arquivos
10. *chmod* - altera os bits do modo de arquivo

4. Comandos de Ajuda e Documentação

5. Comandos de Comunicações

6. Comandos de Edição de Texto

7. Exibição ou Impressão de Arquivos

1. *cat* - Mostra o conteúdo de um arquivo, como o comando type do MD-DOS

8. Comandos de Transferência de Arquivos

1. *rsync* - Sincroniza de forma rápida e flexível dados entre dois computadores.

9. Comandos de Notícias ou Rede

1. *netstat* - Imprime conexões de rede, tabelas de roteamento, estatísticas de interface, conexões mascaradas e associações multicast.
- 2.

10. Comandos de Controle de Processos

11. Comandos de Informação de Estado

12. Comandos de Processamento de Texto

13. Web

1. *html2ps* - Conversor de html para ps

2. [latex2html](#) - Conversor de LaTeX para html
3. [lynx](#) - Navegador web baseado em modo de texto
4. [Chromium](#) - É um projeto de navegador web de código aberto desenvolvido pela Google
5. [sitecopy](#) - Aplicação que nos permite manter fácil e remotamente web sites
6. [weblint](#) - Verificador de sintaxes e de estilos html

14. Comandos para manutenção de hardware

1. [stress-ng](#) - Stressando MEM, DISCO e CPU
2. [cinnamon-settings info](#): Mostra as informações do sistema operacional e de hardware na *interface gráfica cinnamon*.
3. [free](#) - Mostra no console a quantidade de memória instalada.
4. [memtester](#) - teste de memória

15. Pasta do Linux

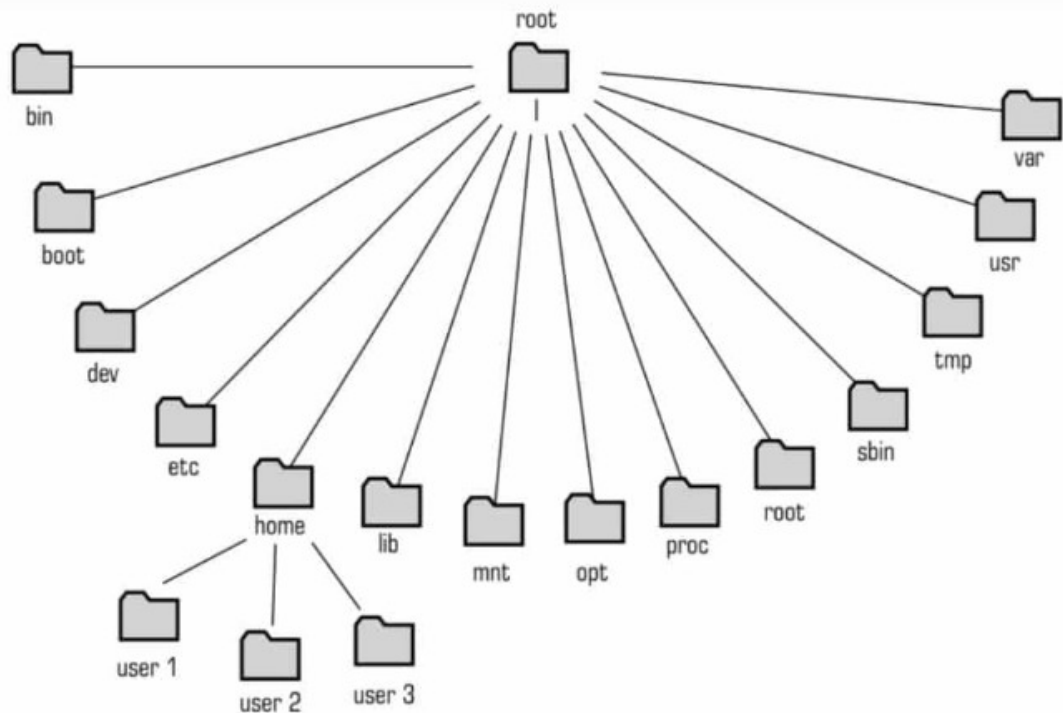
1. Para que serve a pasta `~/local/share` no Linux?
2. ,

16. REFERÊNCIAS

CONTEÚDO

1. Introdução:

- Este documento descreve com exemplos os comandos que costumo usar no dia a dia para a manutenção do sistema.
- A estrutura do sistema de arquivos do Linux é definida por um padrão de mercado chamado Filesystem Hierarchy Standard ou FHS, criado pela comunidade Linux em 1994 e mantida pela Linux Foundation. Atualmente a FHS está na versão 3.0. As distribuições não são obrigadas a seguir este padrão, mas elas entendem a importância da localização dos arquivos e diretórios padronizados. [Veja mais...](#)



- Para compreender a expressão *nome do arquivo fonte* ou *nome do arquivo destino* é necessário seguir a sintaxe de nome de arquivo para a arquitetura linux.
- Use qualquer caractere na página de código atual para um nome, incluindo caracteres Unicode e caracteres no conjunto de caracteres estendido (128–255), exceto os seguintes caracteres porque são interpretados de forma especial:
 - **< (menor que)**
 - Apesar de permitido, não deve ser usado porque é usado por aplicativos tipo shell para redirecionar entrada, quando o aplicativo aceita entrada padrão;
 - **> (maior que)**
 - Apesar de permitido, não deve ser usado porque é usado para *redirecionar a saída* de um comando;
 - **: (dois-pontos)**
 - Apesar de permitido, o mesmo deve *ser evitado* porque o outros sistemas operacionais no qual o linux interage usa como separador de dispositivo ;
 - **" (aspas duplas)**
 - Apesar de permitido, não deve ser usado, porque é usado para separar o nome do arquivo, quando o mesmo tem caracteres especiais no nome do arquivo;
 - **/ (barra)**
 - Não é permitido porque é usada como separar o caminho na árvore de diretórios;

- | (**barra vertical ou pipe**)
 - Apesar de permitido no nome de arquivo, o mesmo deve ser evitado, porque define redirecionamento de software em aplicativos tipo shell;
 - ? (**ponto de interrogação**)
 - Apesar de permitido deve ser evitado porque o shell usa como caractere coringa que representa qualquer caractere na posição onde o mesmo apareça na linha de comandos de aplicativos tipo shell;
 - * (**Asterisco**)
 - Apesar de permitido no linux, o mesmo deve ser evitado, porque é usado como coringa, ou seja: representa uma sequência de caracteres antes ou depois que ele aparece no nome do arquivo nos parâmetros de entrada dos programas tipo shell.
 - . (**Ponto**)
 - É permitido, porém deve ser evitado porque quando usado na primeira posição o arquivo será interpretado como invisível, outros sistemas operacionais que o linux interage usa o ponto para separar o nome do arquivo e o nome da extensão do arquivo;
 - " " (**Espaço em branco**)
 - É permitido, mas como também é um separador de parâmetros de linha de comando, deve-se delimitar o nome com aspas (") para diferenciar dos parâmetros dos aplicativos tipo shell. [Veja a vídeo aula...](#)
- O comprimento de uma especificação de arquivo é limitado.
 - O comprimento máximo de um nome de arquivo é *255 bytes*. O comprimento máximo combinado de nome de arquivo (nome de todas as pastas anteriores a pasta no qual o arquivo pertença) é de *4096 bytes*. Esse comprimento corresponde ao *PATH_MAX* que é suportado pelo sistema operacional. A representação *Unicode* de um caractere pode ocupar *vários bytes*, de modo que o número máximo de caracteres que englobam um caminho e nome de arquivo possa variar. A limitação real é o número de bytes nos componentes de caminho e arquivo, que pode corresponder a um número igual de caracteres.
 - Para operações de recuperação e arquivamento, o comprimento máximo que pode ser especificado para um caminho e nome de arquivo (combinado) permanecerá em *1024 bytes*.
 - **Importante:**
 - Os caracteres `<>|:()&#?*` apesar de aceitos, normalmente precisam ser "escapados" com barra invertida, ou delimitados com aspas em linha de

comando:

- Os caracteres *espaço em branco* e os *caracteres acentuados* apesar de serem permitidos, é bom evitar, porque quando o arquivo é visualizado em outro idioma, esses caracteres podem aparecer de forma errada se a página de código do gerenciador de arquivo for diferente da página de código no qual o mesmo foi criado.
- Apesar de permitir quase todos os caracteres, é bom usar somente os seguintes caracteres abaixo para evitar incompatibilidade quando se usa o sistema de arquivos *NTFS*:
 - a-z
 - A-Z
 - 0-9
 - underline (_)
 - traço (-)
 - ponto (.)

○ **Exemplo de nome de arquivos:**

- `./bkp.sh`
- `../bkp.sh`
- `~/bkp.sh`
- `/home/paulosspacheco/bkp.sh`

2. Atalho para criar um terminal de comandos:

1. Teclas `Ctrl+Alt+T`

3. Comandos de Gestão de Arquivos e Pastas

1. *ls* - O comando *ls* é usado para listar arquivos ou diretórios (pastas):

1. Sintaxe: `$ ls -opções`

```
# EXEMPLOS

# Lista nome dos arquivos, nome dos links e nome das
# pastas em várias colunas
ls

# Lista o atributos, nome do dono, nome do grupo,
# data, hora e nome do arquivo.
ls -l
```

2. *which* - Mostra o nome do arquivo e local (pasta) onde o arquivo foi localizado

1. Sintaxe: `$ which nome_arquivo_no_path`

```
# EXEMPLO
```

```
# Mostra o nome da pasta onde o jva está instalado.  
which java  
> /usr/lib/jvm/java-1.11.0-openjdk-amd64/bin/java
```

▪ **Nota:**

- Só procura nas pastas registrados em `/etc/environment` ou `~/.bashrc`.

3. **cd** - O comando **cd** é usado para mover-se entre os diretórios ou seja: sair de um diretório e ir para o destino passado pelo parâmetro:

1. Sintaxe: `$ cd NomeDaPastaDestino`

```
# EXEMPLOS
```

```
# move-se para a pasta ~/ ou seja: Pasta raiz do  
usuário.  
cd
```

```
# Move-se para a pasta passado por NomeDir.  
# Para nome dia igual ~ (Pasta raiz do usuário) então:  
cd ~
```

```
# Move para a pasta /usr/lib  
cd cd /usr/lib
```

4. **ln** - O comando **ln -s** cria um links simbólico para simplificar o acesso a arquivos que estão com nome da pasta muito grande ou esteja em um hd diferente.

1. **Sintaxe:** `ln -s "target file" "Symbolic filename"`

```
# EXEMPLO:
```

```
# Cria o link pastaRaisDoUsuarioLogado para a pasta ~/  
ln -s ~/ pastaRaisDoUsuarioLogado
```

```
# Criar o link lib para a pasta /usr/lib  
ln -s /usr/lib lib
```

```
# Cria o link environment.txt para o arquivo ~/.bashrc  
ln -s ~/.bashrc environment.txt
```

2. **Notas:**

1. O segundo parâmetro "*target file*" precisa ser um nome de arquivo ou pasta válido;
2. O terceiro parâmetro "*Symbolic filename*" precisa ser um nome diferente de vazio e não pode existir um arquivo ou pasta com mesmo nome do link a ser criado;
3. Ao apagar um link os dados apontados por ele não serão apagados;
4. Caso o link seja copiado para uma pasta de nome diferente da pasta no qual o mesmo foi criado, esse link se torna inválido;
5. O modo de acesso ao link criado, será: *leitura*, *gravação* e *execução* para o *usuário dono*, *grupo* e *outros usuários*, porém o modo de acesso para onde ele aponta, não é alterado.

5. **cp** - O comando **cp** copia um arquivo ou pasta para outra pasta ou arquivo.

1. **Parâmetros:**

1. O atributo do arquivo é levado para a cópia;
2. **-R** : jj
3. **-r** : Copia diretórios recursivamente;
4. **-i** : Pergunta se quer sobrescrever caso o arquivo exista.
 1. Caso exista pressione y para confirmar ou enter para não gravar;
5. **-uprR** : Usado para atualizar o diretório destino com os novos arquivos do diretório de origem.;
6. **-p** : Usado para preservar quando possível o atributo de origem;
7. **-v** : (verbose) explica o que está sendo feito

```
# EXEMPLOS

# Copia o arquivo ~/.bashrc para o arquivo
~/.bashrc.txt
cp ~/.bashrc ~/.bashrc.txt

# Copia o arquivo ~/.bashrc para a pasta ~/tmp
cp ~/.bashrc ~/tmp

# copiar diretórios recursivamente
sudo cp -R /etc ~/tmp

# Copia a toda a pasta /etc e subpastas para a
pasta ~/tmp perguntado se
# quer subscrever caso algum arquivo já exista.
# Caso exista pressione y para confirmar ou enter
para não gravar
sudo cp -i -R /etc ~/tmp

# O parâmetro -f é usado caso um arquivo de
destino existente não puder ser aberto, remove-o e
tente novamente (redundante se a opção -n for usada
cp -f ~/.bashrc ~/tmp

# Atualiza o diretório ~/tmp com os arquivos
diferentes da pasta /etc criado depois da ultima
cópia. Cópia incremental.
# Resumindo: Só será copiado os arquivos com data
superior a data da cópia ou se a cópia não existe o
arquivo de origem.
sudo cp -upvR /etc ~/tmp
```

2. Script [bkp.sh](#) para fazer backup incremental, onde é passado nome de origem (param 1) e destino (param 2):

```
#!/bin/bash
# Program bkp.sh
# Exemplo de uso
# $ ./bkp.sh /Documentos /tmp

origem="$1"
destino="$2"

echo "Atualiza o diretório $destino com os arquivos
diferentes da "
echo "pasta $origem criado depois da ultima cópia."
echo "Cópia incremental".
sudo cp -upvR $origem $destino

result_cp="$?"
if [ $result_cp != 0 ]; then
echo "algo errado na cópia"
exit 1;
fi
```

3. O exemplo de uso do script acima para copiar a pasta ~/Documentos para o destino passado pelo parâmetro 1

```
# Cópia a pasta Documentos para a pasta temporária na
área do usuário.
./bkp.sh ~/Documentos ~/tmp
```

4. O script [bkpdocs.sh](#) executa a mesma tarefa que o script do item 3 acima.

```
#!/bin/bash
# Program bkpDocs.sh
# Exemplo de uso:
#   ./bkpdocs.sh /PastaDestino

origem="~/Documentos"
destino="$1"
if [-z $destino ]; then
echo "O destino precisa ser informado"
exit 1
fi

echo "Atualiza o diretório $destino com os arquivos
diferentes da "
echo "pasta $origem criado depois da ultima cópia."
echo "Cópia incremental".
sudo cp -upvR $origem $destino

result_cp="$?"
if [ $result_cp != 0 ]; then
echo "Algo errado na cópia"
exit 1;
fi
```

5. [Vídeo aula sobre o comando cp...](#)

6. - **dd** : Clona um dispositivo de bloco, convertendo e formatando de acordo com os operandos. [Veja mais...](#)

1. Parâmetros:

1. *'if=/dev/sdax'* : Representa o disco de origem a ser copiado;
2. *'of=/dev/sdbx'* : Representa o disco de destino que vai receber a cópia;
3. *'bs=4096'* : Representa o tamanho do bloco (número de bytes a serem lidos/gravados de cada vez).
4. *conv=noerror,sync* representa o parâmetro de conversão.
 1. A opção *'noerror'* : permite que a ferramenta continue copiando os dados mesmo que encontre erros.
 2. E a opção *'sync'* : permite usar operações de Entrada/Saída sincronizadas.
5. *'status=progress'* : Mostra em que posição está a cópia.

2. Exemplos de backup de todo o disco rígido para uma outra unidade:

```
# 'if=/dev/sda' : representa o disco de origem a ser copiado;  
# 'of=/dev/sdb' : representa o disco de destino que vai receber a cópia;  
sudo dd if=/dev/sda of=/dev/sdb bs=4096  
conv=noerror,sync status=progress
```

3. Exemplos de backup da partição *sda1* para *sdb1*:

```
# Copia a partição sda1 para sdb1 mantendo a mesma formatação.  
sudo dd if=/dev/sda1 of=/dev/sdb1 status=progress  
  
# Copia a partição sda1 para sdb1 usando os parâmetros 4096 e noerror,sync:  
sudo dd if=/dev/sda1 of=/dev/sdb1 bs=4096  
conv=noerror,sync status=progress
```

4. Exemplos de backup da partição *sda1* para o arquivo *sda1.iso*:

```
# Copia a partição sda1 para sdb1 mantendo a mesma formatação.  
sudo dd if=/dev/sda1 of=sdb1.iso status=progress  
  
# Copia a partição sda1 para sdb1 usando os parâmetros 4096 e noerror,sync:  
sudo dd if=/dev/sda1 of=sdb1.iso bs=4096  
conv=noerror,sync status=progress
```

7. **e2image** - Clona um dispositivo de bloco cujo o formato de arquivo seja: *ext2*, *ext3* e *ext4*

1. O objetivo do programa *e2image* é salvar metadados críticos do sistema de arquivos *ext2/ext3/ext4* em um arquivo de imagem do dispositivo.
2. Para clonar uma partição não montada usa-se as opções *-ra* e *-p*. Veja o exemplo de como clonar a partição não montada */dev/sdb1* para */dev/sdb2*:

```
# A opção -a copia os arquivos também além  
sudo e2image -ra -p '/dev/sdb1' '/dev/sdb2'
```

8. **gparted** - Programa usado para criar partição, pode ser usado para clonar partição de forma muito simples e visual.

1. Para clonar uma partição siga os seguintes passos:

1. Instale o programa *gparted* com o comando abaixo:

```
# Atualiza a lista do apt-get
sudo apt update

# Instala o programa gparted
sudo apt install gparted
```

2. Executa o programa *gparted* com o comando abaixo

9. *mount* - **Monta um sistema de arquivos**

1. Exemplo de montagem de pasta compartilhada usando o protocolo samba do windows:

1. **Com domínio:**

```
# mount -t cifs <ORIGEM> <DESTINO> -o user=
<USUARIO>,password=<SENHA>,domain=<DOMINIO>
mount -t cifs //192.168.15.3/publico /mnt/backup/
-o user=paulospacheco,password=1234,domain=itms
```

2. **Sem domínio:**

```
# mount -t cifs <ORIGEM> <DESTINO> -o user=
<USUARIO>,password=<SENHA>
mount -t cifs //192.168.15.3/publico /mnt/backup/
-o user=paulospacheco,password=1234
```

2. **Notas:**

1. *mount*: Comando utilizado para realizar montagens no Linux.
2. *-t*: Tipo de montagem. No nosso exemplo para acessar as pastas compartilhadas no Windows, utilizamos o tipo cifs.
3. *origem*: O caminho de origem do compartilhamento de rede. Ex: `//maquina01/publico` ou `//192.168.50.3/publico`.

4. *destino*: O ponto de montagem onde será montado o compartilhamento Ex: /mnt/backup.
5. *-o*: Parâmetro utilizado para passar outras opções ao comando mount como por exemplo:
 1. *user*: Nome do usuário do domínio que tem permissão de leitura e gravação na pasta de origem compartilhada.
 2. *password*: Senha do usuário informado.
 3. *domain*: Nome do Domínio em que a máquina de origem esta inserida.

3. Exemplo de montagem de pasta compartilhada usando o protocolo NFS:

1. Usando o usuário logado na máquina local e no servidor com mesmo login e senha:

```
# sudo mount <ORIGEM> <DESTINO>
# sudo mount host_ip:/var/nfs/share
~/LnxMint_Server_Share
sudo mount 192.168.15.3:/var/nfs/share
~/LnxMint_Server_Share
```

2. Notas:

1. *mount*: Comando utilizado para realizar montagens no Linux.
2. *origem*: O caminho de origem do compartilhamento de rede.
Ex: 192.168.15.3:/var/nfs/share.
3. *destino*: O ponto de montagem onde será montado o compartilhamento Ex: ~/LnxMint_Server_Share

10. *chmod* altera os bits do modo de arquivo

1. Sintaxe:

1. *chmod -R ZZZ (+ ou - pu) YYY ./*

1. -R significa:

1. Informa ao *chmod* para altera o flag de todas as subpasta dentro da pasta corrente (./)

2. Z significa: Tipo de usuários, onde:

1. *u* = para usuário proprietário do arquivo
2. *g* = para grupo usuários de acesso ao arquivos
3. *o* = para outros usuários não registrados no sistema
4. *a* = para todos os usuários

3. Y significa: tipo de operação, onde:

1. *x* = Execução

2. *r* = Leitura3. *w* = Escrita**2. Exemplo:**

```
# Permite que usuários, grupo e outros executem, leem e gravem todos arquivos da pasta.  
sudo chmod -R ugo+xrw ./
```



```
# Não permite em todos arquivos da pasta, que usuários, grupo e outros executem e gravem os arquivos .  
sudo chmod -R ugo-xw ./
```



```
# Permite em todos arquivos da pasta e subpastas que outros usuários só podem ler os arquivos.  
sudo chmod -R o=r ./
```



```
# Permite em todos arquivos da pasta e subpastas que grupo de usuários só podem ler e gravar os arquivos.  
sudo chmod -R g=rw ./
```

11. *setuid* : Define a identidade do usuário12. *chgrp*: Mudar o grupo de um arquivo ou diretório13. *chown*: Mudar o dono ou grupo de um arquivo ou diretório, vem de change owner14. *chgrp*: Mudar o grupo de um arquivo ou diretório15. *cmp*: Compara dois arquivos16. *comm*: Seleciona ou rejeita linhas comuns a dois arquivos selecionados

17. *crypt*: Encripta ou Descripta arquivos (apenas CCWF)

18. *diff*: Compara o conteúdo de dois arquivos ASCII

19. *file*: Determina o tipo de arquivo

20. *grep*: Procura um arquivo por um padrão, sendo um filtro muito útil e usado, por exemplo um `cat a.txt | grep ola` irá mostrar-nos apenas as linhas do arquivo `a.txt` que contenham a palavra "ola"

21. *gzip*: Comprime ou expande arquivo

22. *ls*: Lista os arquivos abertos, vem de `list open files`

23. *mkdir*: Cria uma diretório, vem de `make directory`

24. *mv*: Move ou renomeia arquivos ou diretórios

25. *pwd*: Mostra-nos o caminho por inteiro da diretório em que nos encontramos em dado momento, ou seja um pathname

26. *quota*: Mostra-nos o uso do disco e os limites

27. *rm*: Apaga arquivos, vem de remove, e é semelhante ao comando del no MS-DOS, é preciso ter cuidado com o comando `rm *` pois apaga tudo sem confirmação por defeito

28. *rmdir*: Apaga diretório, vem de remove directory

29. *stat*: Mostra o estado de um arquivo, útil para saber por exemplo a hora e data do último acesso ao mesmo

30. *sync*: Faz um flush aos buffers do sistema de arquivos, sincroniza os dados no disco com a memória, ou seja escreve todos os dados presentes nos buffers da memória para o disco

31. *sort*: Ordena, une ou compara texto, podendo ser usado para extrair informações dos arquivos de texto ou mesmo para ordenar dados de outros comandos como por exemplo listar arquivos ordenados pelo nome

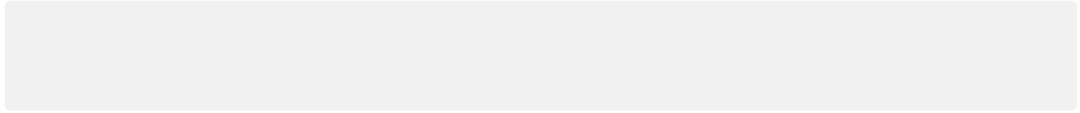
32. *tar*: Cria ou extrai arquivos, muito usado como programa de backup ou compressão de arquivos

33. *tee*: Copia o input para um standard output e outros arquivos

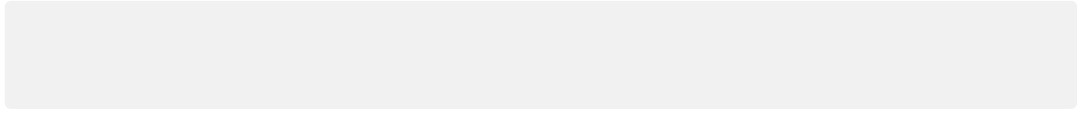
1. O exemplo abaixo verificará a linha do arquivo `file1.txt`, gerar o resultado no terminal e salvar no `file2.txt`.

```
wc -l file1.txt | tee file2.txt
```

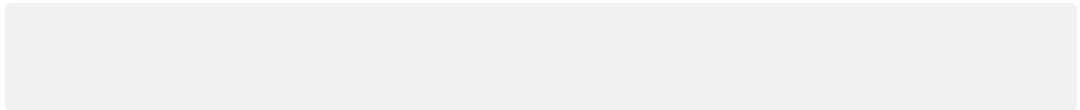
34. *tr*: Traduz caracteres



35. *umask*: Muda as proteções de arquivos



36. *uncompress*: Restaura um arquivo comprimido



37. *uniq*: Reporta ou apaga linhas repetidas num arquivo

38. *wc*: Conta linhas, palavras e mesmo caracteres num arquivo

```
wc -l file1.txt | tee file2.txt
```

39. **Redirecionares**

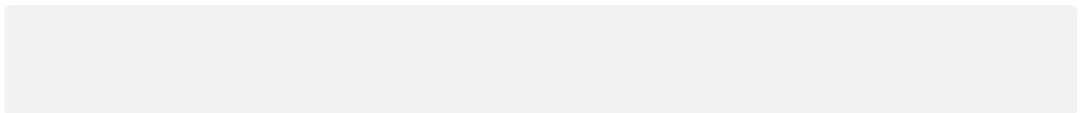
1. `>>` Transfere a saída de um comando para a entrada do comandos após ele.

```
ls -l >> ""Lista de arquivo.txt""
```

1. [Vídeo aula sobre o comando >>...](#)

2. ...

40. *apropos*: Localiza comandos por pesquisa de palavra-chave



41. *find*: Localiza arquivos, como por exemplo: *find . -name *.txt -print*, para pesquisa de arquivos de texto a partir da pasta atual

1. Sintaxe

1. `find ./ -type d -iname 'NomeDaPasta'`

```
# find <startingdirectory> <opções> <termo de busca>  
find ./ -type d -iname 'shell'
```

2. Para configurar o comando *finddir* para que faça pesquisa a partir da pasta atual:

1. Editar o arquivo *~/.bashrc*

```
Editar o arquivo  
xed ~/.bashrc
```

2. No final do arquivo adicionar a linha a seguir:

```
alias finddir="find './' -type d -iname"
```

3. Referências

- [Comandos Linux: Find e Locate – Como Usar](#)

42. *info*: Abre o explorador de informações

43. *Install* : Copie arquivos e defina atributos, copie arquivos enquanto define seus modos de permissão e, se possível, seu proprietário e grupo.

1. Sintaxe:

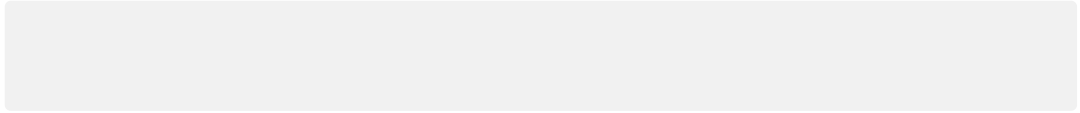
1. install [options]... SOURCE DEST
2. install [options]... SOURCE... DIRECTORY
3. install -d [options]... DIRECTORY...

4. Comandos de Ajuda e Documentação

1. *man*: Manual muito completo, pesquisa informação acerca de todos os comandos que necessitemos de saber, como por exemplo *man find*

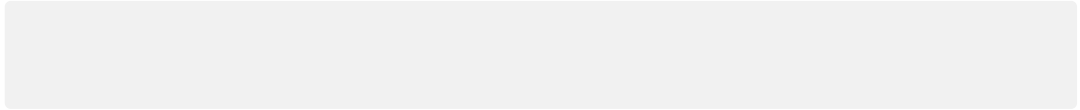
2. *whatis*: Descreve o que um determinado comando é/faz

3. *whereis* : Localizar a página de ajuda (man page), código fonte, ou arquivos binários, de um determinado programa

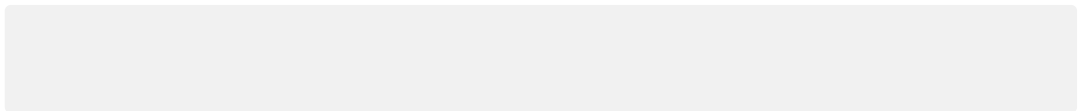


5. Comandos de Comunicações

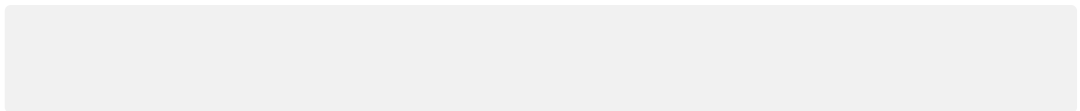
1. *mail*: Enviar e receber emails



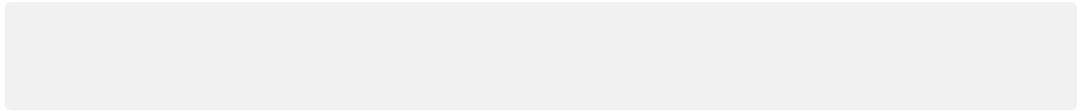
2. *mesg*: Permitir ou negar mensagens de terminal e pedidos de conversação (talk requests)



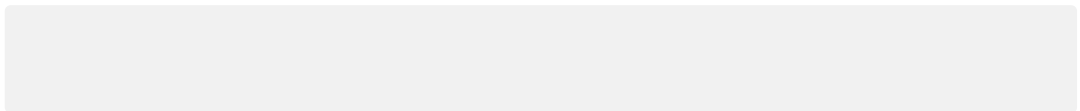
3. *pine*: Outra forma de enviar e receber emails, uma ferramenta rápida e prática



4. *talk*: Falar com outros utilizadores que estejam logados no momento

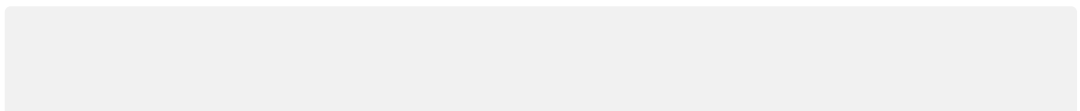


5. *write*: Escrever para outros utilizadores que estejam logados no momento

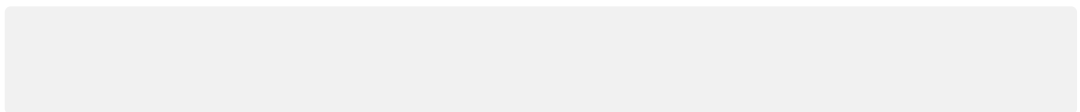


6. Comandos de Edição de Texto

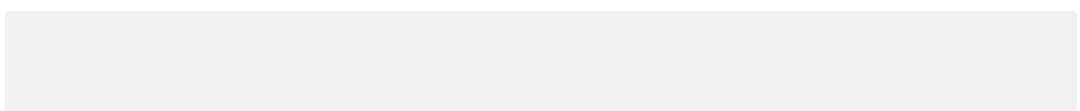
1. *emacs*: Editor de texto screen-oriented

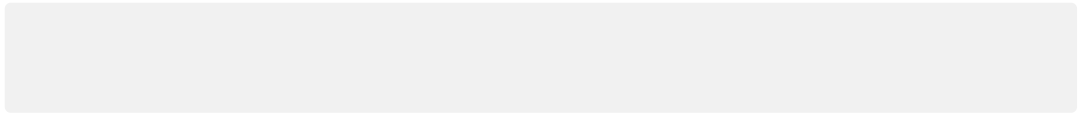
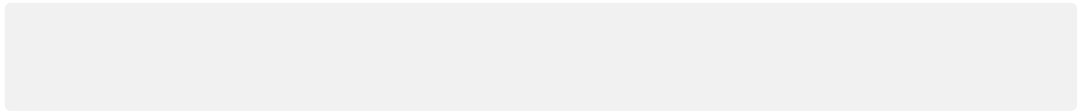


2. *pico*: Editor de texto screen-oriented, também chamado de nano



3. *sed*: Editor de texto stream-oriented



4. *vi*: Editor de texto full-screen5. *vim*: Editor de texto full-screen melhorado (vi improved)

7. >Exibição ou Impressão de Arquivos

1. *cat*: Mostra o conteúdo de um arquivo, como o comando type do MD-DOS.

1. Exemplo 01

1. concatenar nome de arquivos arquivos:

```
cat a.txt b.txt > c.txt
```

2. Exemplo 02

1. O comando abaixo realiza uma série de operações com texto. Vou explicar cada parte em notas abaixo:

```
# Move-se para a pasta /var/lib/dpkg/triggers/  
cd /var/lib/dpkg/triggers/  
  
cat File | cut -d" " -f 2 | uniq -c
```

1. Notas

1. *cat File*: Isso basicamente imprime o conteúdo do arquivo chamado "File" no terminal. O comando *cat* é usado para concatenar e exibir arquivos de texto.
2. *cut -d" " -f 2*: Este comando corta o texto usando o espaço como delimitador (*-d" "*) e seleciona o segundo campo (*-f 2*). Portanto, ele extrai o segundo item de cada linha do texto.
3. *uniq -c*: Este comando remove linhas duplicadas consecutivas do texto e conta quantas vezes cada linha única aparece. O flag *-c* exibe o número de ocorrências de cada linha única.
4. Assim, juntando todas as partes, o comando completo faz o seguinte:
 1. Lê o conteúdo do arquivo "File".
 2. Extrai o segundo campo de cada linha (presumivelmente as linhas estão separadas por

2. *fold*: Encurta, ou seja, faz um fold das linhas longas para caberem no dispositivo de output

3. *head*: Mostra as primeiras linhas de um arquivo, como por exemplo com `head -10 a.txt`, ou usado como filtro para mostrar apenas os primeiros x resultados de outro comando

4. *lpq*: Examina a spooling queue da impressora

5. *lpr*: Imprime um arquivo

6. *lprm*: Remove jobs da spooling queue da impressora

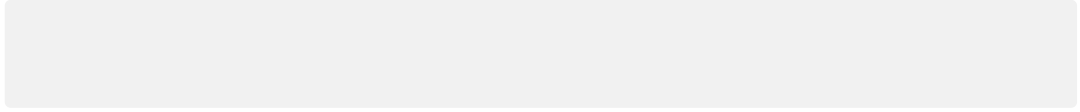
7. *more*: Mostra o conteúdo de um arquivo, mas apenas um ecrã de cada vez, ou mesmo output de outros comandos, como por exemplo `ls | more`

8. *less*: Funciona como o *more*, mas com menos features, menos características e potenciais usos

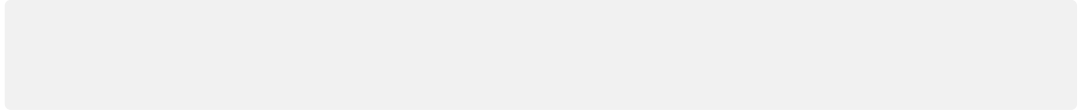
9. *page*: Funciona de forma parecida com o comando *more*, mas exibe os ecrãs de forma invertida ao comando *more*

10. *pr*: Pagina um arquivo para posterior impressão

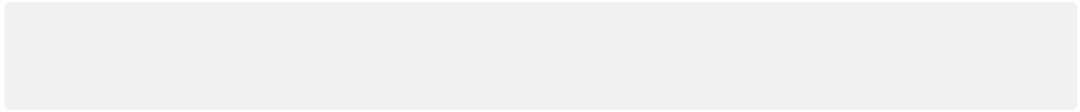
11. *tail*: Funciona de forma inversa ao comando *head*, mostra-nos as últimas linhas de um arquivo ou mesmo do output de outro comando, quando usado como filtro



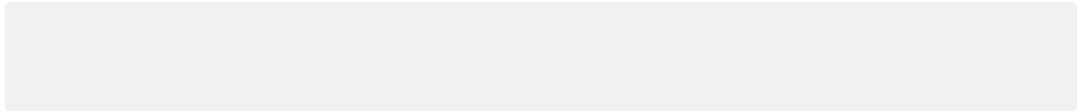
12. *zcat*: Mostra-nos um arquivo comprimido



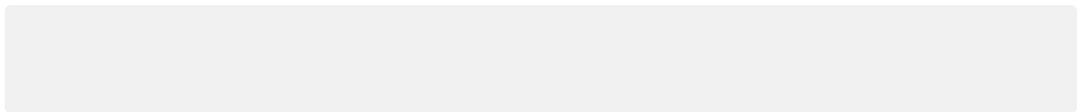
13. *xv*: Serve para exibir, imprimir ou mesmo manipular imagens



14. *gv*: Exibe arquivos ps e pdf

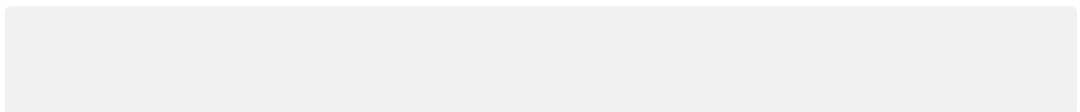


15. *xpdf*: Exibe arquivos pdf, usa o *gv*



8. Comandos de Transferência de Arquivos

1. *ftp*: Vem de file transfer protocol, e permite-nos, usando o protocolo de transferência de arquivos ftp, transferir arquivos entre vários hosts de uma rede, como a um servidor de ftp para enviar ou puxar arquivos



2. - *rsync*: Sincroniza de forma rápida e flexível dados entre dois computadores. [Veja mais...](#)

1. Sintaxe:

1. Cópia para máquina local:

1. *rsync* [parâmetros] [source] [destination]

2. Cópia para máquina remotas:

1. *rsync* [parâmetros] [source] [user]@[host]:[destination]

2. Parâmetros:

1. *-a* : --archive O modo de arquivamento é igual a *-rlptgoD* (sem *-H,-A,-X*).

2. *-r* : Copia a pasta ~/duplicate/ para a pasta ~/original/* recursivamente
3. *-R* : Copia deve ter o nome original e não o nome relativo
4. *-z* : Comprime os dados dos arquivos antes de enviá-los
5. *-v* : Mostra a versão do *rsync* ao iniciar a cópia.
6. *-h* : output num formato legível para humanos.
7. *-l* : Quando links simbólicos forem encontrados, recrie o link simbólico no destino.
8. *-i* : Copia links simbólicos como links simbólicos.
9. *-u* : Força o *rsync* a ignorar todos os arquivos existentes no destino e cuja hora de modificação seja mais recente que o arquivo de origem.
10. *-n* : Modo “dry run” – executa uma tentativa de copiar dados sem realmente copiar qualquer arquivo.
11. *-P* ou *--progress* : Esta opção diz ao *rsync* para imprimir informações mostrando o progresso da transferência.
12. *-p* : Preserva as permissões.
13. *-t* : Preserva datas
14. *-g* : Preserva o grupo.
15. *-o* : Preserva o dono. Precisa de root.
16. *-D* : *--devices --specials* Preservar dispositivos (somente superusuário) +arquivos
17. *--delete* : Exclui arquivos no diretório de destino se eles não existirem no diretório de origem.

```
# Copia todas as pastas do local ~/original/* para a
pasta ~/duplicate/
# -a = -rlptgoD
rsync -zahui --progress ~/original/* ~/duplicate/

# Copia todas as pastas do local ~/original/* para a
pasta ~/duplicate/
# -a = -rlptgoD
# -R = Copia deve ter o nome original e não o nome
relativo
# --delete : Exclui arquivos no diretório de destino se
eles não existirem no diretório de origem.
rsync -zaRhui --progress --delete ~/original/*
~/duplicate/
```

3. Script *copyto*:

1. O script *copyto* copia para o DESTINO, somente os arquivos diferentes, ou os que a data de ORIGEM seja inferior a data de DESTINO. Copia também os arquivos e pastas invisíveis.

```
#!/bin/bash
# Program copyto.sh
# Exemplo de uso:
# ./copyto.sh /PastaDestino
/arquivos_de_exessão.txt

echo "Script: copyto.sh"
echo .
origem_invisivel="./.[^.]*"
origem="/*"
destino="$1"
except="$2"
if [ -z destino ]; then
    echo "O destino precisa ser informado"
    exit 1
fi
echo .
echo "Origem.: $destino"
echo "Destino.: $origem"
echo "Exessão.: $except"
echo "Nota....: Cópia para o DESTINO, somente os
arquivos diferentes, ou os que a data de ORIGEM seja
inferior a data de DESTINO."
echo .
echo "Cópia incremental".
sudo rsync --exclude-from=$except --delete -
arRvhui --progress $origem_invisivel $destino
sudo rsync --exclude-from=$except --delete -
arRvhui --progress $origem $destino
result_cp="$?"
if [ $result_cp != 0 ]; then
    echo .
    echo algo errado na cópia
    exit 1;
fi
```

4. Referências:

1. [rsync_options](#)
2. [10 exemplos do comando rsync para backup e sincronismo de arquivos no Linux](#)
3. [Rsync: confira como utilizar o comando do Linux](#)
4. [Rsync: excluir diretório](#)
5. [Exemplos de Rsync no Linux](#)
6. [Linux / Unix Rsync Copiar apenas arquivos e diretórios de pontos ocultos](#)

3. [scp: Versão segura do rcp](#)

9. Comandos de Notícias ou Rede

1. *netstat*: Imprime conexões de rede, tabelas de roteamento, estatísticas de interface, conexões mascaradas e associações multicast.

1. Exemplo:

```
netstat -an | grep :80
netstat -an | grep :8080
```

2. Notas:

1. *netstat -a*

1. Mostre o estado de todos os soquetes; normalmente os soquetes usados pelos processos do servidor não são mostrados.

2. *netstat -n*

1. Não resolva endereços numéricos e números de porta em nomes.

3. Referência:

1. [netstat\(8\) - página de manual do Linux](#)

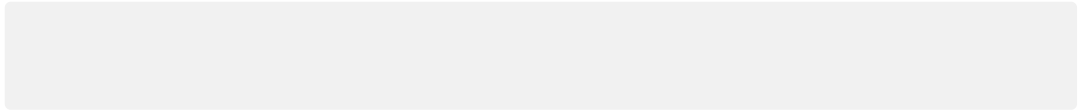
2. *rsh*: Um shell em outros sistemas UNIX

3. *ssh*: Versão segura do rsh

4. *nmap*: Poderoso port-scan, para visualizarmos portas abertas num dado host

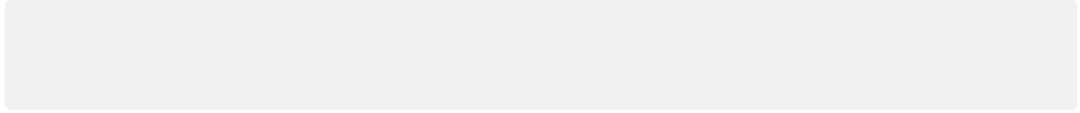
5. *ifconfig*: Visualizar os ips da nossa máquina, entre outras funções relacionadas com ips

6. *ping*: Pingar um determinado host, ou seja, enviar pacotes icmp para um determinado host e medir tempos de resposta, entre outras coisas

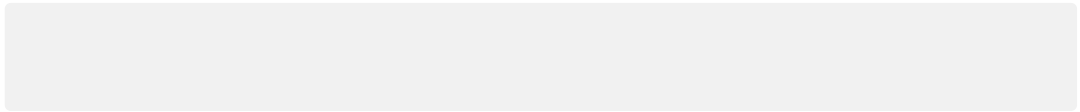


10. Comandos de Controle de Processos

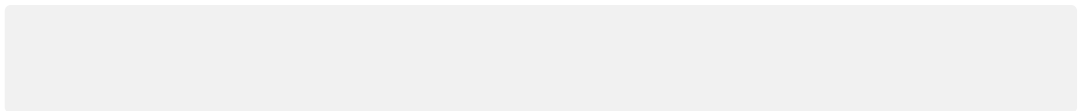
1. *kill*: Mata um processo, como por exemplo `kill -kill 100` ou `kill -9 100` ou `kill -9 %1`



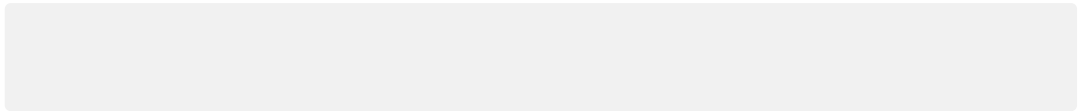
2. *bg*: Coloca um processo suspenso em background



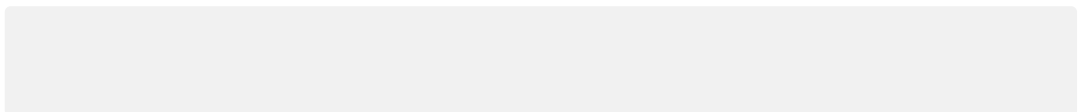
3. *fg*: Ao contrário do comando *bg*, o *fg* traz de volta um processo ao foreground



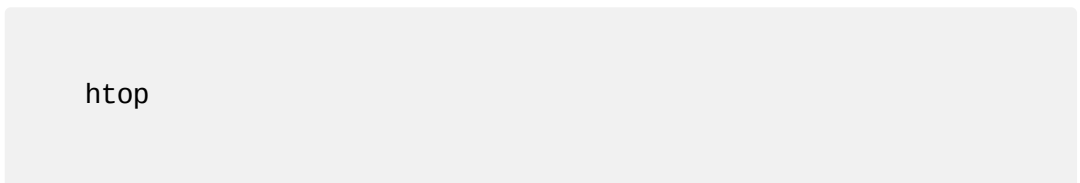
4. *jobs*: Permite-nos visualizar jobs em execução, quando correremos uma aplicação em background, poderemos ver esse job com este comando, e termina-lo com um comando `kill -9 %1`, se for o job número 1, por exemplo



5. *top*: Lista os processos que mais cpu usam, útil para verificar que processos estão a provocar um uso excessivo de memória, e quanta percentagem de cpu cada um usa em dado momento

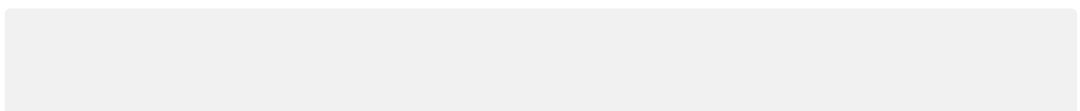


6. *htop* : A mesma finalidade o *top* ao listar os processos ativos, porém contém mais recursos, podendo até matar um processo.



- [Vídeo aula sobre o comando htop...](#)

7. *^y*: Suspende o processo no próximo pedido de input



8. ^z: Suspende o processo actual

11. Comandos de Informação de Estado

1. *clock*: Define a hora do processador

2. *date*: Exibe a data e hora

3. *df*: Exibe um resumo do espaço livre em disco

```
df -h
```

4. *blkid* : utilitário de linha de comando para localizar/imprimir atributos do dispositivo de bloco.

```
blkid
```

5. *du*: Exibe um resumo do uso do espaço em disco

6. *ncdu* : Lista em ordem decrescente quais diretórios estão usando o espaço em disco.

```
ncdu
```

- [Vídeo aula sobre o comando ncdu...](#)

7. *mc* Midnight Commander : Gerenciador de arquivos modo texto. Útil quando operando o servidor não possa usar modo gráfico.

```
ncdu
```

- [Vídeo aula sobre o comando mc...](#)

8. *env*: Exibe as variáveis de ambiente

9. *finger*: Pesquisa informações de utilizadores

10. *history*: Lista os últimos comandos usados, muito útil para lembrar também de que comandos foram usados para fazer determinada acção no passado ou o que foi feito em dada altura

11. *last*: Indica o último login de utilizadores

12. *lpq*: Examina a spool queue

13. *manpath*: Mostra a path de procura para as páginas do comando man

14. *printenv*: Imprime as variáveis de ambiente

15. *ps*: Lista a lista de processos em execução, útil para saber o pid de um processo para o mandar abaixo com o comando

16. *kill*, entre outras coisas

17. *pwd*: Mostra-nos o caminho por inteiro do diretório em que nos encontramos em dado momento, ou seja um pathname

18. *set*: Define variáveis da sessão, ou seja, da shell, na C shell, na bash ou na ksh

19. *spend*: Lista os custos ACITS UNIX até à data

20. *time*: Mede o tempo de execução de programas

21. *uptime*: Diz-nos há quanto tempo o sistema está funcional, quando foi ligado e o seu uptime

22. *w*: Mostra-nos quem está no sistema ou que comando cada job está a executar

23. *who*: Mostra-nos quem está logado no sistema

24. *whois*: Serviço de diretório de domínios da Internet, permite-nos saber informações sobre determinados domínios na Internet, quando um domínio foi registado, quando expira, etc

25. *whoami*: Diz-nos quem é o dono da shell

12. Comandos de Processamento de Texto

1. *abiword*: Processador de Texto Open Source
2. *addbib*: Cria ou modifica bases de dados bibliográficas
3. *col*: Reverte o filtro a line feeds
4. *diction*: Identifica sentenças com palavras
5. *diffmk*: Marca diferenças entre arquivos
6. *dvips*: Converte arquivos TeX DVI em arquivos PostScript
7. *explain*: Explica frases encontradas pelo programa diction
8. *grap*: Preprocessador pic para desenhar gráficos, usado em tarefas elementares de análises de dados
9. *hyphen*: Encontra palavras com hífen
10. *ispell*: Verifica a ortografia de forma interativa
11. *latex*: Formata texto em LaTeX, que é baseado no TeX
12. *pdfelateX*: Para documentos LaTeX em formato pdf
13. *latex2html*: Converter LaTeX para html
14. *lookbib*: Encontra referências bibliográficas
15. *macref*: Cria uma referência cruzada listando arquivos de macros nroff/troff
16. *ndx*: Cria uma página de indexação para um documento
17. *neqn*: Formata matemáticas com nroff
18. *nroff*: Formata texto para exibição simples
19. *pic*: Produz simples imagens para troff input
20. *psdit*: Filtra um output troff para a Apple LaserWriter
21. *ptx*: Cria uma indexação permutada mas não em CCWF
22. *refer*: Insere referências de bases de dados bibliográficas
23. *roffbib*: Faz o run off de uma base de dados bibliográfica
24. *sortbib*: Ordena uma base de dados bibliográfica
25. *spell*: Encontra erros de ortografia
26. *style*: Analisa as características superficiais de um documento
27. *tbl*: Formata tabelas para nroff/troff
28. *tex*: Formata texto
29. *tpic*: Converte arquivos pic source em comandos TeX
30. *wget*: Permite-nos fazer o download completo de páginas web, com todos os arquivos, de forma fácil e não interactiva, sem exigir por isso presença do utilizador, respeitando também o arquivorobots.txt

13. Web

1. *html2ps*: Conversor de html para [ps](#)
2. *latex2html*: Conversor de LaTeX para html
3. *lynx*: Navegador web baseado em modo de texto, ou seja, é um web browser que nos permite abrir todo o tipo de páginas visualizando apenas os textos elinks, não vendo assim as imagens, e sendo por isso bastante rápido, mas requiere prática para ser manuseado
4. *Chromium* é um projeto de navegador web de código aberto desenvolvido pela Google, no qual o Google Chrome baseia o seu código-fonte.
5. *sitecopy*: Aplicação que nos permite manter fácil e remotamente web sites

14. Comandos para manutenção de hardware

1. *stress-ng* : Estressando MEM, DISCO e CPU

1. Com a ferramenta Stress-ng podemos literalmente realizar o Stress de alguns recursos do seu servidor, sendo eles: Cpu, memória e disco. Com o Stress-ng podemos estressar nosso servidor afim de descobrir o quanto de carga ele aguentará sem perder a performance. É possível fazer testes isolados nos componentes informados, para a análise de desempenho ser mais assertiva.

1. O stress-ng criado para fazer uma máquina trabalhar com alto consumo para simular problemas de hardware como excedentes térmicos, bem como erros do sistema operacional que ocorrem apenas quando um sistema é golpeado com força.

2. INSTALANDO STRESS-NG

```
sudo apt update  
sudo apt install stress-ng
```

3. Exemplo de uso:

1. Recomendado pelo desenvolvedor do projeto:

1. O comando abaixo é bem completo, o que cada opção faz está documentado no comando *stress-ng -h*

```
sudo stress-ng --cpu 1 --vm 1 --hdd 1 --fork  
1 --switch 1 --timeout 10 --metrics
```

2. Executar 8 tarefas de memória virtual que combinados usam 80% da memória disponível por 1 hora. Assim, cada tarefa usa 10% da memória disponível:

```
sudo stress-ng --vm 8 --vm-bytes 80% -t 1h
```

3. Executa por 60 segundos com 4 tarefas de CPU, 2 tarefas IO e 1 node usando 1G de memória virtual:

```
sudo stress-ng --cpu 4 --io 2 --vm 1 --vm-  
bytes 1G --timeout 60s
```

4. Executa 2 instâncias das tarefas de IO combinados usando um total de 10% do espaço disponível no sistema de arquivos por 10 minutos. Cada tarefa utilizará 5% do espaço disponível no sistema de arquivos:

```
sudo stress-ng --iomix 2 --iomix-bytes 10% -t  
10m
```

5. Com este comando é possível medir as latências de agendamento em tempo real criadas pelo workers do disco rígido. O comando usa um relógio de alta resolução em nanossegundos para medir as latências durante o sleeps de 10.000 nanossegundos. No final de 1 minuto de tensão, a distribuição de latência com intervalos de 2500 ns será exibida. OBS: deve ser executado com privilégios de superusuário para permitir que o agendamento em tempo real obtenha medidas precisas:

```
sudo stress-ng --cyclic 1 --cyclic-dist 2500  
--cyclic-method clock_ns --cyclic-prio 100--  
cyclic-sleep 10000 --hdd 0 -t 1m
```

2. [cinnamon-settings info](#): Mostra as informações do sistema operacional e de hardware na *interface gráfica cinnamon*.

1. Este comando pode ser encontrado no menu de opções do cinnamon com o nome de *Informações do Sistema* ou pode ser executado pelo console da seguinte forma:

```
cinnamon-settings info
```

1. Nota : Este comando não pode ser executado como root porque o root não pode executar o browser da web.

3. [free](#) - Mostra no console a quantidade de memória instalada.

```
free -m
```

4. *memtester* - teste de memória

Checa o tamanho da memória para saber o total de memória a ser testada.

```
free -m
```

	total	usada	livre	compart.
buff/cache disponível				
Mem.:	7883	2575	1945	186
3363	4826			
Swap:	0	0	0	

Testa uma vez o total de memória.

```
memtester 7883 1
```

Loop 1/1:

Stuck Address	: ok
Random Value	: ok
Compare XOR	: ok
Compare SUB	: ok
Compare MUL	: ok
Compare DIV	: ok
Compare OR	: ok
Compare AND	: ok
Sequential Increment	: ok
Solid Bits	: ok
Block Sequential	: ok
Checkerboard	: ok
Bit Spread	: ok
Bit Flip	: ok
Walking Ones	: ok
Walking Zeroes	: ok
8-bit Writes	: ok
16-bit Writes	: ok

Done.

15. PASTAS DO LINUX

1. Para que serve a pasta `~/.local/share` no Linux?

1. A pasta `~/.local/share` no Linux é uma localização onde aplicativos podem armazenar dados específicos do usuário. Isso pode incluir configurações, caches, e outros dados que são específicos para cada usuário do sistema. Muitos aplicativos usam essa pasta para manter dados que não são essenciais para o funcionamento do aplicativo, mas que podem melhorar a experiência do usuário ou acelerar o desempenho do aplicativo ao

armazenar informações em cache. Por exemplo, navegadores da web muitas vezes armazenam caches de páginas da web nessa pasta para que as páginas possam ser carregadas mais rapidamente quando acessadas novamente. Além disso, alguns aplicativos podem armazenar configurações do usuário nessa pasta para personalizar o comportamento do aplicativo para cada usuário. Em resumo, a pasta `~/local/share` é usada por aplicativos no Linux para armazenar dados específicos do usuário que não se encaixam em outras categorias, como configurações ou caches.

16. REFERÊNCIAS

1. [Manual Linux](#)
2. <https://www.hostinger.com.br/tutoriais/como-criar-link-simbolico-linux/>
3. <https://canaltech.com.br/linux/aprenda-a-usar-links-simbolicos-e-hardlinks-no-linux/>
4. [cp = Copy one or more files to another location.](#)
5. [30 comandos mais usados do Linux](#)
6. [Comandos Importantes Linux](#)
7. [10 exemplos do comando rsync para backup e sincronismo de arquivos no Linux](#)
8. [rsync_options](#)
9. [Estressando MEM, DISCO e CPU com Stress-ng \(Debian9\)](#)

HISTÓRICO

- 23/04/2024
 - Escrever o texto para que serve a pasta `~/local/share` no Linux?
- 23/05/2023 a 25/05/2023
 - ☐ Catalogar os principais comandos do sistema operacional linux.
- 22/05/2023
 - ☒ Concluir documento: introdução descrevendo como o documento foi criado.
- 19/05/2023
 - ☒ Criado documento do comando `cp`;
 - ☒ Iniciar o documento: introdução descrevendo como o documento foi criado.
- 18/05/2023
 - ☒ Criado documento do comando `ln` (link simbólico)
- 17/03/2023
 - ☒ Criado documento dos comandos `ls` e `cd`;
- 15/05/2021
 - ☒ Criado este documento;

